

Trabalho 1 - Parte 1

Aplicação de “bate-papo distribuído”

Sistemas Distribuídos (ICP-367)

Profa. Silvana Rossetto

¹IC/CCMN/UFRJ

Introdução

O objetivo deste trabalho é **projetar um aplicação de “bate-papo distribuído”** para aplicar os conceitos estudados até aqui.

O trabalho deverá ser feito em grupos de 3 ou 4 alunos.

Nessa Parte 1, faremos o projeto da solução (com artefato de entrega). Para a Parte 2, teremos a implementação, avaliação e apresentação da aplicação.

Para o contexto deste exercício, um **bate-papo distribuído** é uma aplicação distribuída que permite que **pares de usuários** remotos troquem mensagens de texto.

Requisitos gerais da aplicação

A solução implementada deverá permitir que o usuário, ao entrar na aplicação, sinalize que está pronto para receber mensagens e tenha acesso à relação de outros usuários que também estão ativos. Daí, o usuário poderá enviar mensagens para qualquer outro usuário ativo e, ao mesmo tempo, receber e responder mensagens de outros usuários. Quando desejar sair da aplicação, o usuário deverá sinalizar que não mais estará ativo.

Não é necessário criar janelas distintas para cada conversa. Pode-se usar, por exemplo, um rótulo no início de cada mensagem para indicar o remetente ou o destinatário da mensagem. Dessa forma, todas as mensagens trocadas podem ser exibidas na mesma janela.

Atividade 1

Objetivo: Projetar a **interface de usuário** da aplicação.

Roteiro: Considere as funcionalidades desejadas listadas abaixo e elabore o menu da aplicação.

1. Indicar que o usuário está ativo (apto a receber mensagens).
2. ~~Indicar que o usuário está inativo (não está apto a receber mensagens).~~
3. Visualizar os usuários que estão ativos.
4. Iniciar e manter uma conversa com um usuário que está ativo.
5. Visualizar as mensagens recebidas de outros usuários.
6. Encerrar a aplicação.

Cada grupo deve descrever aqui como será a interface com o usuário.

Atividade 2

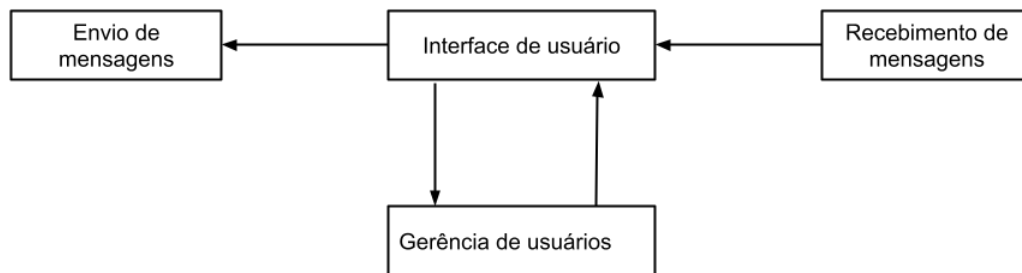
Objetivo: Projetar a **arquitetura de software** da aplicação.

Roteiro: Considerando as funcionalidades listadas acima, projete os **componentes da aplicação** (funcionalidades e interfaces). Use os estilos arquiteturais estudados como ponto de partida.

Resposta geral: Os componentes principais da aplicação serão:

- **Interface de usuário:** exibe as funcionalidades da aplicação para o usuário e as mensagens recebidas;
- **Gerência de usuários:** registra, mantém e disponibiliza a lista dos identificadores únicos dos usuários;
- **Envio de mensagens:** envia uma mensagem de texto para um usuário de destino registrado (ativo);
- **Recebimento de mensagens:** recebe uma mensagem de texto de um usuário fonte registrado (ativo).

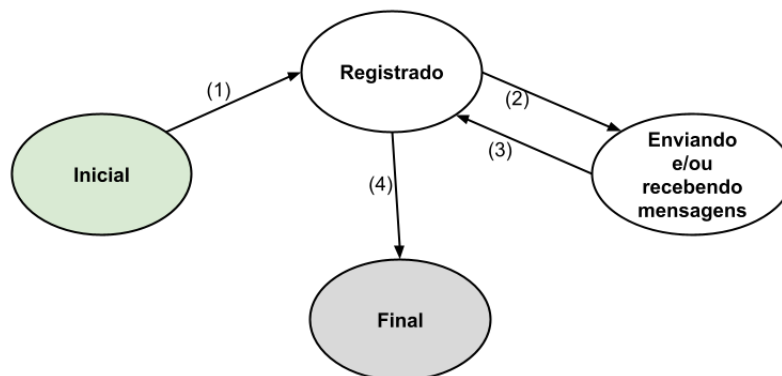
A Figura ?? abaixo mostra os componentes e suas possíveis iterações.



Os estados principais da aplicação serão:

- **Inicial:** aplicação iniciada;
- **Registrado (ativo):** aplicação apta a enviar e receber mensagens;
- **Enviando e/ou recebendo mensagens:** aplicação enviando e ou recebendo mensagens;
- **Final:** aplicação finalizada.

A Figura ?? abaixo mostra os estados da aplicação e suas possíveis transições.



As transições (1) e (4) ocorrem apenas uma vez, enquanto as transições (2) e (3) podem ocorrer diversas vezes. Na transição (1), um identificar único de usuário e sua

localização devem ser passados para realizar o seu registro. Na transição (4), o registro do usuário deve ser apagado. Na transição (2), o usuário de destino e a mensagem devem ser indicados quando a operação de envio é selecionada.

Atividade 3

Objetivo: Projetar a **arquitetura de sistema** da aplicação.

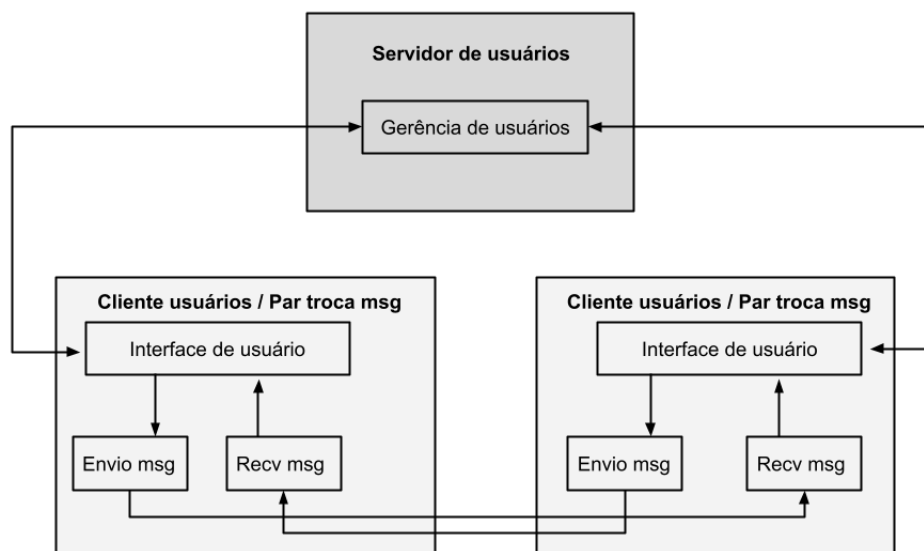
Roteiro: Escolha uma das arquiteturas de sistema estudadas (cliente-servidor, par-a-par, híbrida) e descreva como a arquitetura de software projetada na Atividade 2 será instanciada em uma arquitetura de sistema, definindo:

1. como os componentes de software serão agrupados ou particionados;
2. onde os componentes de software serão implantados;
3. como os componentes de software deverão interagir.

Resposta geral:

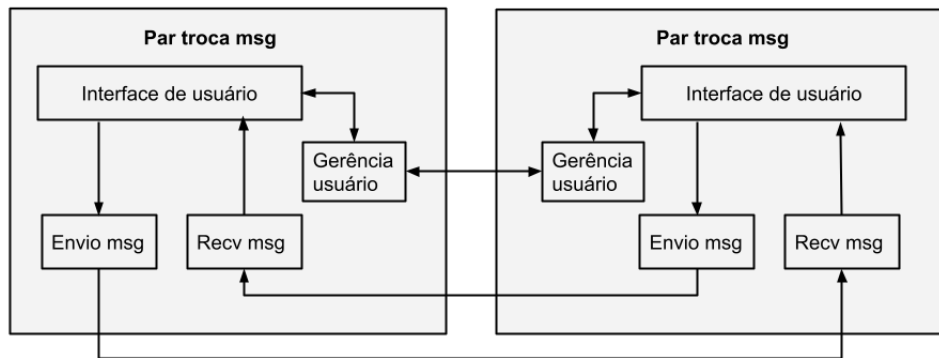
Proposta de arquitetura de sistema híbrida (**solução escolhida**)

Em uma possível arquitetura híbrida, a gerência de usuários fica a cargo de um nó separado (servidor) e os demais componentes ficam nos demais nós da aplicação (clientes e pares). A figura abaixo mostra essa proposta de arquitetura de sistema híbrida:



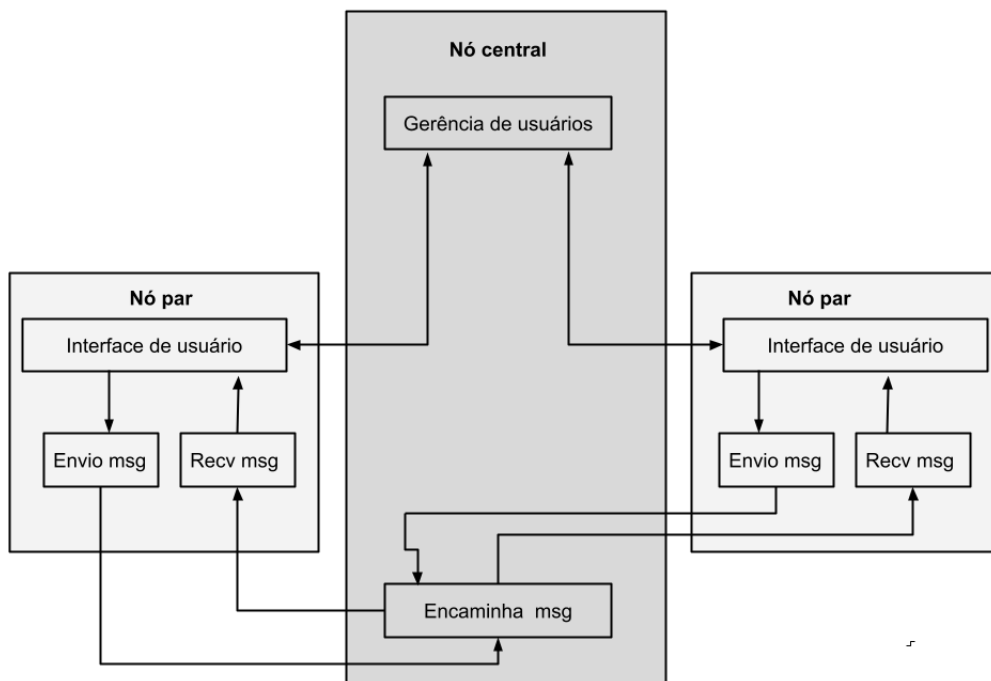
Proposta de arquitetura de sistema par-a-par

Em uma possível arquitetura par-a-par, a gerência de usuários fica a cargo de realizar a descoberta de nós e manter o registro de usuários ativos. A figura abaixo mostra essa proposta de arquitetura de sistema par-a-par:



Proposta de arquitetura de sistema centralizada

Em uma possível arquitetura centralizada, a gerência de usuários e toda a troca de mensagens ficam a carga de um nó central. Os demais nós fazem suas requisições de registro e envio de mensagens para o nó central; e recebem por meio dele as mensagens para as quais o nó é o destinatário. A figura abaixo mostra essa proposta de arquitetura de sistema centralizada:



Atividade 4

Objetivo: Projetar o **protocolo de camada de aplicação** da aplicação.

Roteiro: Projete um **protocolo de camada de aplicação**, considerando as decisões tomadas nas Atividades 2 e 3, definindo:

1. protocolo de camada de transporte;
2. tipos de mensagens trocadas (ex., requisição, resposta);
3. sintaxe/formato de cada tipo de mensagem (campos da mensagem com seus tamanhos e tipos de dados);
4. regras que determinam quando e como um processo envia/responde mensagens.

Pode-se optar por usar algum formato de troca de mensagem (ex., JSON).

Resposta geral:

1. protocolo de camada de transporte: **TCP**
2. tipos de mensagens trocadas (ex., requisição, resposta):
 - (a) requisição cliente-servidor para registro de usuário
 - (b) resposta servidor-cliente sobre registro de usuário
 - (c) requisição cliente-servidor de lista de usuários
 - (d) resposta servidor-cliente de lista de usuários
 - (e) requisição cliente-servidor para encerramento
 - (f) resposta servidor-cliente para encerramento
 - (g) envio de mensagem de texto da aplicação
3. sintaxe/formato de cada tipo de mensagem (campos da mensagem com seus tamanhos e tipos de dados): Determinamos que as **mensagens usarão o formato JSON**, com a determinação do tipo de operação explícita em um dos atributos. Esse JSON será enviado e recebido como uma string. Com relação à **identificação do tamanho da resposta**, determinamos que o **primeiro byte** da mensagem identificará a quantidade de bytes da mensagem completa para podermos capturá-la sem erro pelo TCP.

Por fim, os campos das mensagens serão:

- **Requisições do Cliente:**

- { “operacao”: “login”, “username”: “luanzinho32”, “porta”: 5000 }
- { “operacao”: “logout”, “username”: “luanzinho32” }
- { “operacao”: “get_lista” }

- **Respostas do Servidor Central:**

- { “operacao”: “login”, “status”: 200, “mensagem”: “Login com sucesso” }
- { “operacao”: “login”, “status”: 400, “mensagem”: “Username em Uso” }
- { “operacao”: “logout”, “status”: 200, “mensagem”: “Logout com sucesso” }
- { “operacao”: “logout”, “status”: 400, “mensagem”: “Erro no Logout” }
- { “operacao”: “get_lista”, “status”: 200, “clientes”: { “luanzinho32”: { “Endereco”: “10.10.10.10”, “Porta”: “5000” }, “ney”: { “Endereco”: “20.20.20.20”, “Porta”: “7776” }, “Usuario”: { “Endereco”: “IP”, “Porta”: “Porta” } } }
- { “operacao”: “get_lista”, “status”: 400, “mensagem”: “Erro ao obter a lista” }

- **Mensagens trocadas entre Usuários:**

- { “username”: “luanzinho32”, “mensagem”: “oi!” } (o username é de quem está enviando a mensagem)
4. regras que determinam quando e como um processo envia/responde mensagens:
- (a) requisição cliente-servidor para registro de usuário: deve ser realizada antes de começar a enviar e receber mensagens
 - (b) resposta servidor-cliente sobre registro de usuário: verifica se já existe outro usuário com o mesmo username ; se sim, não altera a lista e retorna com aviso de “usuário em uso”; se não, inclui o usuário na lista de usuários e retorna com aviso de “Login com sucesso”.
 - (c) requisição cliente-servidor de lista de usuários: deve ser realizada para receber a lista atual de usuários
 - (d) resposta servidor-cliente de lista de usuários: retorna a lista atual completa de usuários
 - (e) requisição cliente-servidor para encerramento: deve ser realizada antes do usuário sair da aplicação
 - (f) resposta servidor-cliente para encerramento: verifica se o usuário está na lista de usuários; se sim, retira o usuário da lista de usuários e envia mensagem de “logoff de sucesso”; se não envia mensagem “erro no logoff”
 - (g) envio de mensagem de texto da aplicação: o usuário deve informar qual é o usuário de destino, se esse usuário estiver na lista de usuários, envia a mensagem para o destino e exibe exceção caso o usuário não esteja ativo no momento; se o usuário não estiver na lista, a mensagem não deve ser enviada

Atividade 5

Objetivo: Projetar a implementação da aplicação.

Roteiro: Descreva em linhas gerais como a aplicação será modularizada e implementada, considerando as decisões tomadas nas Atividades 1, 2, 3 e 4.

Entrega Gere um arquivo PDF com as **respostas/decisões das Atividades 1 e 5 (não precisa mais repetir as respostas das demais atividades)**. Inclua a identificação de todos os alunos do grupo (nome e DRE) e envie o arquivo pelo Google Sala de Aula (uma resposta é suficiente para o grupo).